UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 SPECIAL TOPIC IN DATA ENGINEERING

SECTION 01

AI ALGORITHM TUTORIAL

LECTURER: DR. ARYATI BINTI BAKRI

GROUP 5

Student Name	Matric No.
GOE JIE YING	A23CS0224
LAM YOKE YU	A23CS0233
TAN YI YA	A23CS0187

1. Image Classification

Image classification is the process of classifying the entire image and assigning it a single label from a predefined set of categories. The main goal is to enable computers to automatically recognize and categorize images in a way similar to human visual recognition. Due to its ability to categorize visual data instantly, image classification is widely applied in many fields such as healthcare, environmental monitoring and autonomous systems.

A common example is the CIFAR-10 dataset, which is used in this tutorial. It contains 10 image categories, including airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships and trucks. When a CNN model is trained on CIFAR-10, it learns to distinguish features from each class and correctly predict the label of each image based on its visual characteristics. For instance, recognizing fur patterns for animals, wheels and vehicle shapes for transportations.

2. CNN

A Convolutional Neural Network (CNN) is a deep learning model designed to process visual and grid-like data. CNN automatically learns and extracts important image features, like edges, corners and textures.

CNN consists of several key components:

- Convolutional Layers: Using filters or kernels, these layers detect features such as edges, textures and more complex patterns in input images.
- Pooling Layers: These layers reduce the dimensions of the input to reduce the computational complexity in the network.
- Activation Functions: Converts extracted feature maps into a one-dimensional vector.
- Fully Connected Layers: These layers are responsible for making predictions based on the high-level features learned and assigns probability scores to different classes.

Figure 1 shows the basic architecture of a CNN for image classification. The input image passes through several convolutional and pooling layers to extract important features before being flattened and processed by fully connected layers to get the final classification result.

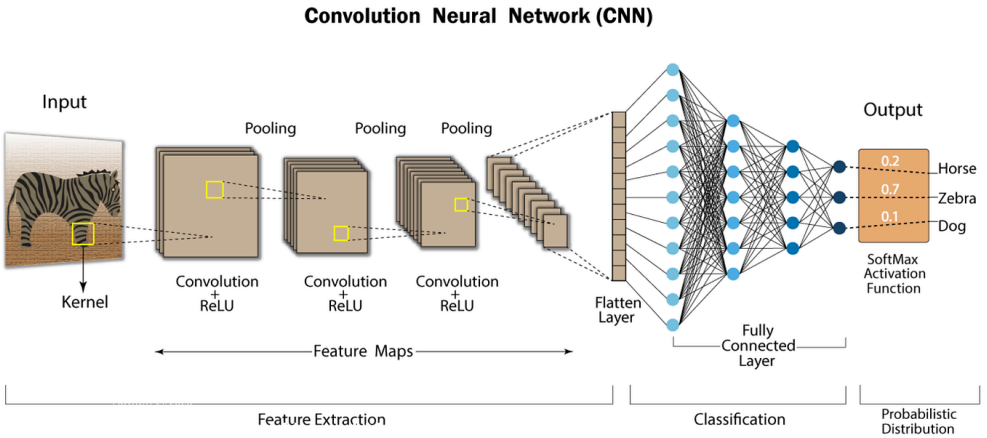

Figure 1: Convolution Neural Network (CNN) Architecture

Source: Adapted from Haque, K. N. (2023), "What is Convolutional Neural Network — CNN (Deep Learning)," Medium.

Through the CIFAR-10 dataset, the CNN model learns to distinguish features using a hierarchical learning process. Initial layers focus on identifying fundamental elements like basic edges and color gradients, whereas subsequent layers are trained to recognize intricate patterns, including textures such as animal fur or specific structures like wheels. Finally, the network utilizes these learned characteristics to predict the appropriate label for each input image.

3. Evaluation of CNN

The performance of a Convolutional Neural Network (CNN) is evaluated using several metrics, including accuracy, loss, precision, recall and F1-score that measure how effectively the model classifies images. These metrics provide insights into the model's learning capability, prediction accuracy and overall classification performance.

3.1 Accuracy

Accuracy is one of the most commonly used evaluation metrics in image classification. It measures the proportion of images correctly categorized by a model out of the total images evaluated. The formula used to calculate the accuracy is as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where:

- *TP = True Positives*
- *TN = True Negatives*
- *FP = False Positives*
- *FN = False Negatives*

A higher accuracy value indicates that the model is able to classify a large portion of images correctly. For image classification tasks such as CIFAR-10 in this tutorial, accuracy provides a general indication of the model's overall performance.

3.2 Loss

Loss measures the difference between the model's predicted outputs and the actual class labels. In this tutorial, CNN used the Adam optimization algorithm to minimize the loss value and Sparse Categorical Crossentropy is used as the loss function because the CIFAR-10 dataset labels are represented as integer values rather than one-hot encoded vectors.

A lower loss value indicates that the model's predictions are closer to the actual labels. In order to determine whether the model is learning effectively and whether issues such as underfitting or overfitting are occurring, monitoring loss throughout training is a good choice.

3.3 Precision

Precision measures the proportion of correctly predicted positive instances among all instances predicted as positive. The formula is listed below:

$$Precision = \frac{TP}{TP + FP}$$

High precision shows that the model makes fewer false positive predictions. In image classification, a high precision value means that when the model predicts a particular class, it is likely to be correct.

3.4 Recall

Recall measures the proportion of actual positive instances that are correctly identified by the model, with the formula listed:

$$Recall = \frac{TP}{TP + FN}$$

A high recall value means the model is effective at recognizing nearly all relevant instances of a specific class. This metric is important when failing to identify a positive instance is costly or undesirable.

3.5 F1-Score

F1-score is the harmonic mean of precision and recall, which is being calculated by following formula:

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

The F1-score provides a balanced measure of model performance, especially when precision and recall are equally important. A higher F1-score represents a better balance between correctly identifying positive instances and minimizing incorrect positive predictions.

3.6 Interpretation of Evaluation Metrics

Each evaluation metric provides different indicators and insights into the performance of the CNN model. Accuracy measures the overall correctness of predictions, while loss tells how well the model is learning by measuring prediction error. Precision evaluates the correctness of positive instances, recall measures the model's ability to identify actual positive cases, whereas F1-score gives balanced evaluation between these two metrics.

4. Steps hands on Python

Step 1: Import Libraries

The necessary libraries for building and training neural networks using TensorFlow and Keras, and for numerical operations and plotting are imported.

```python
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
```

```
import numpy as np
import matplotlib.pyplot as plt
```

Step 2: Load Dataset

The CIFAR-10 dataset is loaded. The dataset consists of 60,000 32x32 color images in 10 classes, with 6,000 images per class.

```
Python
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
```

Step 3: Verify and Reshape Dimensions

The shapes of the feature and labels are inspected. Training labels (y_train) and testing labels (y_test) are reshaped into 1D arrays for Keras loss function input. The first 5 labels of the training labels are displayed before and after the transformation to verify the structural effects of the reshape operation.

```
Python
print("X_train shape:", X_train.shape)
print("X_test shape :", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_train (before reshape):", y_train[:5])

# Perform the reshapes
y_train = y_train.reshape(-1,)
y_test = y_test.reshape(-1,)

print("y_train (after reshape) :", y_train[:5])
```

Step 4: Map Output to Text

A list of class names that correspond to the integer labels in the CIFAR-10 datasets is defined to map numerical output to text for interpretation.

```
Python
classes = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog',
'horse', 'ship', 'truck']
```

Step 5: Define plot_sample Function

Function plot_sample is defined to use image data (X), labels (y), and index to display a single image with its corresponding class label. This function is used to check training dataset and evaluate training results in a later stage.

```
Python
def plot_sample(X, y, index):
```

```
    plt.figure(figsize=(15, 2))
    plt.imshow(X[index])
    plt.xlabel(classes[y[index]])
```

Step 6: Check Training Data

The plot_sample function is called to verify the validity of training data.

```
Python
plot_sample(X_train, y_train, 5)
plt.show()
plot_sample(X_train, y_train, 501)
plt.show()
```

Step 7: Data Normalization

Data normalization is performed on feature data by scaling the pixel values of training feature (X_train) and testing feature (X_test) from the range [0, 255] to [0, 1] to help neural networks learn more effectively.

```
Python
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Step 8: ANN

Artificial Neural Network (ANN) model is defined, compiled and trained on the training data for 5 epochs. Keras's Sequential API is used to stack individual network layers in a linear sequence. The model is compiled with SGD optimizer, sparse_categorical_crossentropy loss, and accuracy as a metric.

```
Python
ann = models.Sequential([
    layers.Flatten(input_shape=(32, 32, 3)),
    layers.Dense(3000, activation='relu'),
    layers.Dense(1000, activation='relu'),
    layers.Dense(10, activation='softmax')
])

ann.compile(optimizer='SGD',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])

ann.fit(X_train, y_train, epochs=5)
```

Step 9: Classification with ANN

The necessary functions for evaluating the model are imported. The trained ANN model is used to predict classes on the test data (X_test). The predictions (probabilities) are converted into discrete class labels using np.argmax. A classification report is printed for viewing the precision, recall, f1-score for each class and the overall accuracy.

```
Python
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print("Classification Report: \n", classification_report(y_test,
y_pred_classes))
```

Step 10: Confusion Matrix for ANN

The seaborn library is imported for statistical data visualization. A confusion matrix for ANN is plotted using seaborn to compare true labels (y_test) with predicted class labels (y_pred_classes).

```
Python
import seaborn as sns

cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=classes,
yticklabels=classes)
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.title('Confusion Matrix for ANN Model')
plt.show()
```

Step 11: CNN

Convolutional Neural Network (CNN) model is defined, compiled and trained on the training data for 10 epochs. Two Conv2D layers with ReLU activation and MaxPooling2D layers are used for feature extraction, followed by a Flatten layer, a Dense hidden layer with ReLU, and a Dense output layer with softmax activation. The model is compiled using the adam optimizer, sparse_categorical_crossentropy loss function, and accuracy as the evaluation metric.

```
Python
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu',
input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),
```

```
    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

cnn.compile(optimizer='adam',
            loss = 'sparse_categorical_crossentropy',
            metrics=['accuracy'])

cnn.fit(X_train, y_train, epochs=10)
```

Step 12: Evaluate CNN

The trained CNN model on the test dataset (X_test, y_test) is evaluated to assess its performance on unseen data. The loss and accuracy are displayed.

```
Python
cnn.evaluate(X_test, y_test)
```

Step 13: Classification with CNN

The trained CNN model is used to make predictions on the testing feature (X_test). Raw probability outputs for the first 5 test images are displayed. Each row represents a test image, and the columns represent the probabilities of belonging to each of the 10 classes.

```
Python
y_pred = cnn.predict(X_test)
y_pred[:5]
```

Step 14: Preview and Compare Predicted and True Label

The raw probability predictions from the CNN model are converted into discrete class labels for each test image. The first 5 predicted class labels are displayed.

```
Python
# Predicted Label
y_classes = [np.argmax(element) for element in y_pred]
print("Predicted label (y_classes[:5]):\n", y_classes[:5])

# Display true labels
print("Actual labels (y_test[:5]):\n", y_test[:5])
```

Step 15: Random Sample Verification using plot_sample

The test image at index 60 is shown along with the true label. Then, the predicted label for the same index is shown using the classes list. This process is repeated multiple times at random index value to imitate random sample verification process manually.

```python
plot_sample(X_test, y_test, 60)
plt.show()

classes[y_classes[60]]
```

Step 16: Confusion Matrix for CNN

A confusion matrix for CNN is plotted using seaborn to compare true labels (y_test) with predicted class labels (y_classes).

```python
cm_cnn = confusion_matrix(y_test, y_classes)
plt.figure(figsize=(10, 8))
sns.heatmap(cm_cnn, annot=True, fmt='d', cmap='Blues', xticklabels=classes, yticklabels=classes)
plt.xlabel('Predicted')
plt.ylabel('Truth')
plt.title('Confusion Matrix for CNN Model')
plt.show()
```

5. Weakness of the current CNN model

Limited Feature Extraction: Only 2 convolution layers implemented, while modern image models usually need deeper hierarchies (edges → textures → shapes → objects). With only 2 layers, the model cannot build rich features

Small fully connected layer: After flattening, the nodes are reduced to just 64 neurons. The model loses learned spatial information too aggressively

No normalisation: This will cause the training to be less stable, slower convergence and lower accuracy.

No regularisation: This may introduce overfitting or underfitting.

Basic pooling strategy: Only MaxPooling after each conv block and no advanced feature reuse or deeper structure

Basic learning strategy: No learning rate schedule or tuning and insufficient epochs

6. Enhanced CNN model

Step 1: Create the model

```python
improved_cnn = models.Sequential([

    # Data augmentation (helps generalization)
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),

    # Block 1
    layers.Conv2D(32, (3,3), padding='same', activation='relu',
input_shape=(32,32,3)),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3,3), padding='same', activation='relu'),
    layers.MaxPooling2D(2),
    layers.Dropout(0.25),

    # Block 2
    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    layers.MaxPooling2D(2),
    layers.Dropout(0.25),

    # Block 3
    layers.Conv2D(128, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2),
    layers.Dropout(0.25),

    # Classifier
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
```

The improvements are as follow:

Data augmentation: The images were randomly changed slightly, such as randomly flip, rotate and zoom the image. This helps the models to generalise better by exposing them to a wider range of variations.

`padding='same'`: A new parameter is added to the Conv2D layer. This keeps the image the same size after convolution, by adding zero-padding around the image. This keeps the image size consistent and does not shrink too fast when going through different layers, plus, the

patterns at the edges are not ignored. Compared to the previous CNN, the padding parameter is not defined, and would be `padding='valid'` by default, which reduces the image from 32x32 to 30x30.

`BatchNormalization()`: As the values inside layers can become inconsistent after calculations, `BatchNormalization()` standardises data inside the network so training becomes faster, more stable and more accurate.

Two Conv2D layers in a row: The repeated Conv2D allows the network to learn more complex features step-by-step. The first Conv2D detects simple patterns and the second Conv2D combines those patterns into more meaningful features. The BatchNormalization is placed between them to stabilise the output of the first Conv2D and makes the second conv learn better.

`Dropout()`: The model randomly disables 25% of neuron output to zero during training so the model does not rely too much on any single one. This regularisation technique reduces overfitting and improves generalisation.

Conv2D layer with 128 features: Increasing the number of filters to 128 allows the network to learn more detailed and higher-level features from the images.

Stronger Dense layer: After flattening, the input neurons are connected to 256 neurons instead of 64 in the previous CNN.

Step 2: Compile the model

```python
improved_cnn.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

The improvements are as follow:
Learning rate: It controls how big a step the model takes when it tries to improve itself. However, the current setting of 0.001 is the same as the default one if not explicitly specified.

Step 3: Train the model

```python
callbacks = [
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=3,
```

```
        verbose=1
    ),
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=7,
        restore_best_weights=True
    )
]

history_improved = improved_cnn.fit(
    X_train, y_train,
    epochs=30,
    validation_split=0.2,
    callbacks=callbacks
)
```

The improvements are as follow:
Callbacks:

- `ReduceLROnPlateau`: If the model stops improving, reduce the learning rate. It monitors validation loss (val_loss), how bad the model is on unseen data. If val_loss is not improving for 3 epochs (`patience=3`), then reduce the learning rate by a factor of 0.5.
- `EarlyStopping`: If the model stops improving for 7 epochs (`patience=7`), stop training. This prevents the model from continuing to train unnecessarily and prevents overfitting. After training stops, go back to the best version of the model (`restore_best_weights=True`).

Epochs: Increased number of epochs to 30

Validation data: Validation data is included to help detect overfitting in the latter sections and supports callbacks. Initially the validation data used was the test data. However, to ensure fairness, the train data is split, with 20% of the data being validation data. This also means that the training data will be reduced, which may reduce the accuracy and increase loss.

7. Evaluation of the ANN, CNN and improved_CNN model

All the models are evaluated using the `evaluate()` function, `classification_report()` function, `confusion_matrix()` function and `summary()`.

7.1 Overall Performance Comparison

The following table summarises the performance of the ANN, CNN and improved CNN models using different evaluation metrics.

	ANN	CNN	Improved CNN

Loss	1.3862	0.8964	0.6429
Accuracy	0.5060	0.7063	0.7855
Precision	0.54	0.70	0.80
Recall	0.51	0.70	0.79
F1-score	0.50	0.70	0.78

Table 1: Performance comparison

According to Table 1, CNN performs better than ANN, achieving lower loss and higher accuracy, precision, recall and F1-score. The improved CNN also performed better than the initial CNN.

7.2 Confusion Matrix and Per-Class Performance

The classification reports and confusion matrices for all models are shown below.

ANN

Classification report for ANN:

	precision	recall	f1-score	support
airplane	0.46	0.73	0.57	1000
automobile	0.65	0.67	0.66	1000
bird	0.50	0.20	0.28	1000
cat	0.43	0.27	0.33	1000
deer	0.31	0.69	0.43	1000
dog	0.50	0.38	0.43	1000
frog	0.69	0.36	0.47	1000
horse	0.60	0.57	0.59	1000
ship	0.66	0.62	0.64	1000
truck	0.59	0.57	0.58	1000
accuracy			0.51	10000
macro avg	0.54	0.51	0.50	10000
weighted avg	0.54	0.51	0.50	10000

Figure 7.2.1 Classification report for ANN

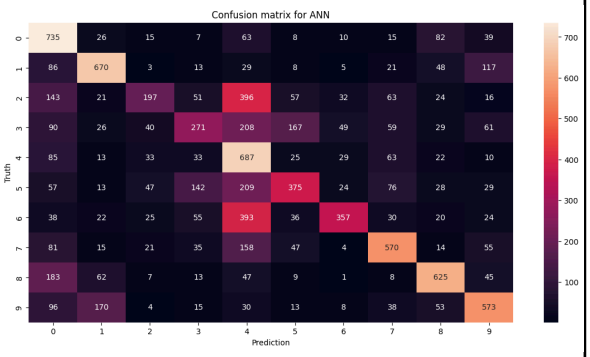

Figure 7.2.2 Confusion matrix for ANN

CNN

Classification report for CNN:

	precision	recall	f1-score	support
airplane	0.70	0.79	0.74	1000
automobile	0.82	0.80	0.81	1000
bird	0.61	0.57	0.59	1000
cat	0.48	0.54	0.51	1000
deer	0.67	0.63	0.65	1000
dog	0.61	0.61	0.61	1000
frog	0.81	0.73	0.77	1000
horse	0.75	0.74	0.75	1000
ship	0.84	0.76	0.80	1000
truck	0.73	0.81	0.77	1000
accuracy			0.70	10000
macro avg	0.70	0.70	0.70	10000
weighted avg	0.70	0.70	0.70	10000

Figure 7.2.3 Classification report for CNN

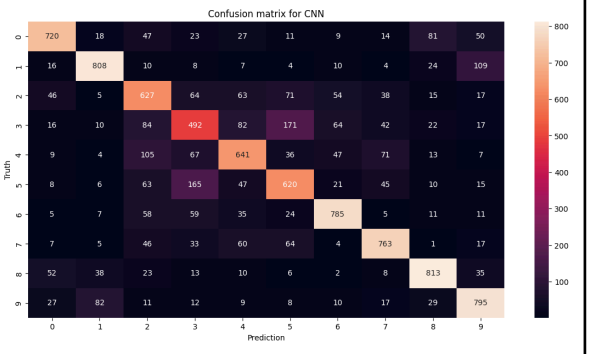

Figure 7.2.4 Confusion matrix for CNN

Improved CNN

Classification report for Improved CNN:

	precision	recall	f1-score	support
airplane	0.83	0.82	0.83	1000
automobile	0.91	0.90	0.90	1000
bird	0.79	0.65	0.72	1000
cat	0.78	0.51	0.61	1000
deer	0.73	0.75	0.74	1000
dog	0.83	0.65	0.73	1000
frog	0.59	0.94	0.73	1000
horse	0.88	0.82	0.85	1000
ship	0.87	0.91	0.89	1000
truck	0.79	0.91	0.84	1000
accuracy			0.79	10000
macro avg	0.80	0.79	0.78	10000
weighted avg	0.80	0.79	0.78	10000

Figure 7.2.5 Classification report for Improved CNN

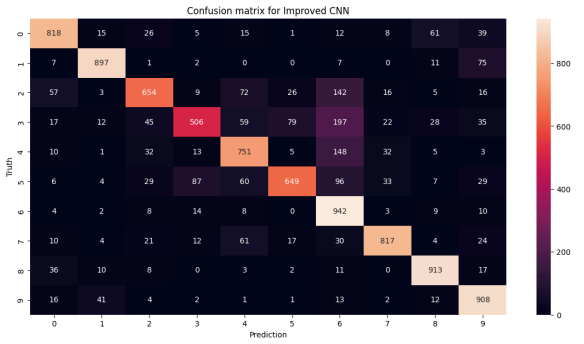

Figure 7.2.6 Confusion matrix for Improved CNN

Table 2: Classification Report and Confusion Matrix

Based on the classification reports and confusion matrices, the animal classes, including bird, cat, deer and dog, generally achieved lower accuracy, precision and F1-scores. These categories are visually similar and contain greater variation in texture, pose and background, which increases the difficulty of classification.

Cats and dogs are frequently confused. In the CNN model, 165 dogs were labelled as cats and 171 cats were labelled as dogs. In the improved CNN, the misclassification was reduced to 87 dogs labelled as cats and 79 cats labelled as dogs.

It is also interesting to note that a noticeable number of animal images were falsely classified as frogs in the improved CNN. This includes 142 birds, 197 cats, 148 deer and 96 dogs being classified as frogs.

In addition, trucks and automobiles were often confused. In the CNN model, 82 trucks were identified as automobiles and 109 automobiles were classified as trucks. In the improved CNN, the misclassification was reduced but remained significant, with 41 trucks classified as automobiles and 75 automobiles classified as trucks.

7.3 Learning Performance and Generalisation of Improved CNN

The improved CNN model was further evaluated using training and validation accuracy and loss curves, given the inclusion of validation data in the training process. The training and validation accuracy and loss were shown in the following figure.

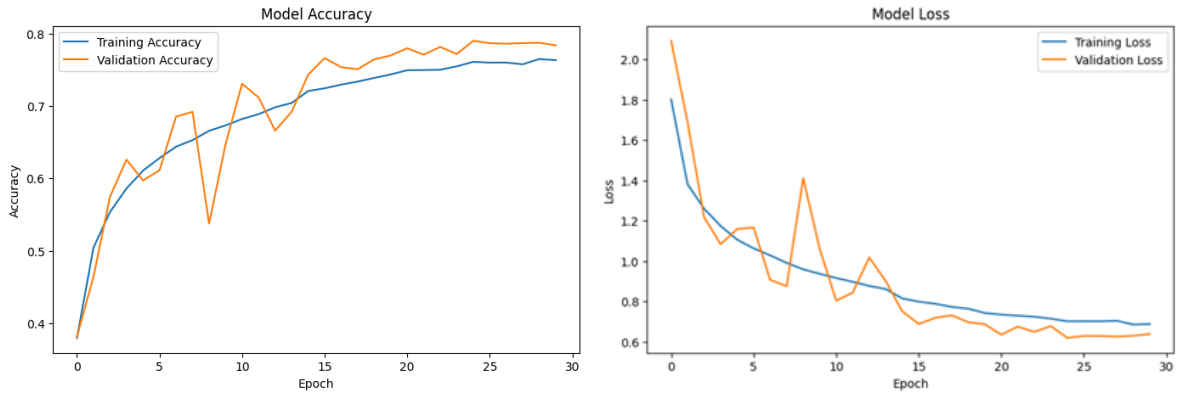

Figure 7.3 Model Accuracy and Model Loss for Improved CNN model

The training and validation accuracy increased steadily across epochs, while both loss values decreased over time. Although some fluctuations were observed between epochs, the overall trend indicates that the model learned effectively during training.

The validation curves remained relatively close to the training curves, suggesting that the model generalised reasonably well without severe overfitting.

7.4 Computational Cost and Trade-off

The computational complexity of the models was analysed using training time, and number of parameters and memory usage obtained from the `summary()` function.

	ANN	CNN	Improved CNN
Epochs	10	10	30
Training Time	8m 8.6s	3m 24.1s	33m 32.6s
Total parameters	12,230,012	502,688	2,003,456
Trainable parameters	12,230,010	167,562	667,498
Memory Usage	46.65 MB	1.92 MB	7.64 MB

Table 3 Computational Cost and Complexity Comparison

Although the improved CNN achieved the best classification performance, it also introduced higher computational complexity compared to the basic CNN model. The improved CNN contained approximately 2 million parameters, compared to 502 thousand parameters in the original CNN, resulting in increased training time and memory usage. In addition, the improved CNN was trained for 30 epochs, compared to 10 epochs for the ANN and basic CNN models, which also contributed to the higher computational cost.

However, despite the ANN model containing the largest number of parameters due to the dense layer, at over 12 million, its performance remained significantly lower than both CNN-based models. This demonstrates that CNN architectures are more parameter-efficient for image classification tasks because convolutional layers can effectively learn spatial features while using fewer parameters through weight sharing.

Overall, the improved CNN performed better than the initial CNN due to the deeper architecture, normalisation layers and dropout regularisation techniques. However, although the improved CNN yields a better result, the training time of the improved CNN is more expensive compared to the other two models.

8. Results and Discussion

The ANN model produced the weakest performance, achieving an accuracy of 50.60%, while the CNN improved the accuracy to 70.63%. The improved CNN achieved the best overall performance with an accuracy of 78.55%, representing an improvement of 11.21% over the original CNN.

CNN-based models are more effective than ANN for image recognition because convolutional layers learn spatial patterns and textures through convolution layers, unlike ANN, which flattens the image pixels directly at the beginning. As shown in Table 3, the ANN contained over 12.2 million parameters and required 46.65 MB of memory, yet achieved only 50.60% accuracy and the highest loss value of 1.3862. This suggests that increasing the number of parameters alone did not improve classification performance of ANN.

8.1 Improvement of Enhanced CNN over CNN Model

The improved CNN introduced additional convolutional layers and increased the number of filters up to 128. This increased the model capacity has improved accuracy from 70.63% to 78.55% and reduced loss from 0.8964 to 0.6429, which indicates that the model was able to learn more discriminative image features.

In addition, Figure 7.3, the training and validation curves of model accuracy and model loss shows that validation accuracy is highly correlated with training accuracy, and same goes for validation loss and training loss. This suggests that the improved CNN maintained stable learning throughout training, despite being trained for three times more epochs than the original CNN, which means that the model generalised well to unseen data.

The confusion matrix revealed that the improved CNN reduced several major classification errors observed in the original CNN. Misclassification between cats and dogs decreased from a total of 336 cases to 166 cases, representing approximately a 51% reduction. Similarly, confusion between automobiles and trucks decreased from a total of 191 cases to 116 cases, a reduction of approximately 39%. These reductions suggest that the enhanced architecture learned more distinctive representations for visually similar classes.

9. Reflection

Lam Yoke Yu

In this tutorial, three models, including an ANN model, a CNN model and an improved CNN model, were built to classify images. The models were trained on the CIFAR-10 training dataset and tested on the test dataset. The results clearly showed that the ANN produced the weakest performance, while the improved CNN model achieved the best results.

Through this tutorial, I learned how these models work and how they perform image classification step by step. The unexpected challenges are the long training time used in the improved CNN, which took 80 minutes when it was trained.

Another challenge faced is the usage of validation data in the enhanced CNN model. At first, the test data was used as validation data, but considering bias that may occur, the validation was adjusted to be the 20% of the training data. This caused the accuracy to drop. Out of curiosity, a model was also trained without validation and callback, and the results were just different in decimals.

Overall, this tutorial was a valuable learning experience. Future work could involve testing the models on different datasets and further evaluating their performance, as different types of data may be better suited to different models.

Goe Jie Ying

Through this tutorial, a practical experience to develop a CNN and also its improved model for image classification were gained using the CIFAR-10 datasets. It improved understanding of how CNN models learn and classify image data. It also shows the importance of improving the model in order to achieve better performance.

A clear difference between ANN, CNN and CNN improved models was observed. The ANN achieved lower accuracy around 50% due to loss of spatial information, CNN performed better with 70% accuracy, while CNN improved model had the highest accuracy of 79% among these three. Therefore, choosing a correct model for a specific purpose is an important skill to learn.

During the development of the CNN model, a challenge was faced, where the relatively moderate performance of the initial CNN model, including signs of overfitting where training accuracy was higher than test accuracy. However, through enhancement and evaluation of the improved models, followed by iterative testing, the challenges were resolved.

Overall, this experience enriched the technical skills in machine learning and provided a better understanding of the importance and implementation specifically to classify image data. Future improvements for this tutorial would be training the model using more advanced architectures such as ResNet or adjusting learning rates and optimizers to improve classification accuracy and performance.

Tan Yi Ya

This hands on tutorial on image classification on the CIFAR10 dataset provided valuable experience in developing and evaluating ANN, CNN and enhanced CNN models. Through the tutorial, a deeper understanding was gained of how convolutional neural networks extract image features and ANN models are not suitable for image classification tasks. The comparison of model performances highlighted the importance of architecture design, feature extraction and regularization techniques to achieve higher accuracy.

One significant challenge encountered was during the initial setup phase when downloading the CIFAR-10 dataset in a local Jupyter Notebook environment. The download process was extremely slow and repeatedly stalled, preventing progress on model development. After investigating possible network and system-related causes, the project was migrated to Google Colab, which resolved the issue immediately and enabled efficient access to cloud computing resources.

For future improvement, transfer of learning approaches using pre-trained models such as ResNet or MobileNetV3 could be explored. Additionally, automated hyperparameter optimization techniques could be applied to further improve model performance and training efficiency.